Computer Vision Group Project

ARI3129

Assignment Report

Daniel Pace, Amy Spiteri, Ellyn Rose Debrincat, Joachim Grech

January 25, 2026

Contents

1 Introduction **1**

2 Background on the techniques used **1**
2.1 Faster R-CNN 1
2.2 YOLO (You Only Look Once) 1

3 Data Preparation **2**

4 Implementation of the object detectors **2**
4.1 COCO-based Detector 2
4.1.1 EfficientDet 2
4.1.2 Faster R-CNN 2
4.1.3 RetinaNet 3
4.1.4 RF-DETR 3
4.2 YOLO-based Detectors 3

5 Evaluation of the object detectors **4**

6 References and List of Resources Used **i**

1 Introduction

The Aim of this project was to implement and evaluate various object detection algorithms on a custom dataset. The project was split up into the following sections:

- Building a dataset:
    1. Image collection,
    2. Image cleaning,
    3. Image annotation,
    4. Data analysis, and
    5. Data preprocessing.
- Object detection/ localization:
    1. Implementation of two object detectors to detect sign type,
    2. Implementation of one object detector to detect a specific sign attribute, and
    3. Evaluation of the trained detectors, including metrics and visualizations.
    4. Analysis of input image, supporting the maintenance and monitoring of traffic signs.

The following steps were taken to ensure an equal and fair distribution of work among all group members:

- For task 1, each group member contributed a similar number of annotated images to the dataset.
- For task 2, each team member was assigned one YOLO-based detector and one COCO-based detector to implement and evaluate.

2 Background on the techniques used

2.1 Faster R-CNN

Faster R-CNN [2] represents a significant advancement in object detection by unifying the region proposal step with the detection network, thereby eliminating the computational bottleneck of external algorithms like Selective Search used in R-CNN [4] and Fast R-CNN [3].

The core innovation is the Region Proposal Network (RPN), a fully convolutional network that shares features with the detection head. The RPN slides a window over the convolutional feature map generated by a backbone such as ResNet [5] to simultaneously predict object bounds and objectness scores at each position. To handle multi-scale detection, it utilizes anchor boxes of various scales and aspect ratios. The entire system is trained end-to-end using a multi-task loss function that combines classification and bounding box regression objectives.

2.2 YOLO (You Only Look Once)

YOLO [6] fundamentally changed the approach to object detection by framing it as a single regression problem, rather than a classification task applied to region proposals. Unlike two-stage detectors like Faster R-CNN, which first generate potential regions and then classify them, YOLO processes the full image in a single pass through the neural network.

The architecture divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that cell is responsible for detecting the object. Each grid cell predicts B bounding boxes and confidence scores for those boxes, along with C class probabilities. This unified architecture allows for extremely fast inference speeds suitable for real-time applications. While early versions (YOLOv1-v3) sometimes struggled with small objects compared to region-based methods, modern iterations (such as the YOLOv8-v12 models used in this project) have incorporated feature pyramids and advanced augmentation techniques to significantly close this gap.

3 Data Preparation

To prepare the dataset for training, the following steps were taken:

1. The images were collected manually from diverse locations on the island, and at different times of the day. The goal was to capture a solid variety of sing conditions, angles, and lighting scenarios.
2. The collected images were then cleaned to remove any duplicates, after which, any recognizable faces or licence plates were obscured to ensure privacy. This was done both manually and using automated tools that can be found in the 'Annotation Process' folder of the GitHub repository [1] (all automated outputs were manually verified before moving onto the next step).
3. The images were then annotated manually using LabelStudio as instructed.
4. todo: about merging datasets
5. The combined dataset was then zipped and uploaded to the shared repository folder for easy access by all group members.

4 Implementation of the object detectors

4.1 COCO-based Detector

The implementation of the different COCO-based detectors varied slightly between models due to their unique architectures and requirements, and having been implemented by different group members. In general, the aim was to match the outputs of the YOLO-based detectors in terms of training performance metrics. The main metric we aimed to optimize was the mAP at IoU=0.5 and at 0.5:0.95. The following COCO-based detectors were implemented:

4.1.1 EfficientDet

4.1.2 Faster R-CNN

The Faster R-CNN implementation was based on the pre-trained `fasterrcnn_resnet50_fpn_v2` model from torchvision, using the FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT weights as a starting point. The model was fine-tuned on the custom traffic sign dataset with 6 sign classes plus a background class.

Model Architecture

The base model utilizes a ResNet-50 backbone with Feature Pyramid Network (FPN) for multi-scale feature extraction. The region of interest (RoI) heads were modified to accommodate the custom number of classes by replacing the box predictor with a FastRCNNPredictor configured for 7 output classes.

Dataset Configuration

The training and validation datasets were loaded using PyTorch's CocoDetection class with images transformed to tensors. A custom anchor generator was configured with five anchor sizes (16, 32, 64, 128, 256 pixels) and three aspect ratios (0.5, 1.0, 2.0) to better detect signs of varying sizes. The training used a batch size of 6 with pin memory enabled for faster GPU data transfer.

Training Configuration

The model was trained for up to 100 epochs with the following hyperparameters:

- Learning rate: 0.005
- Momentum: 0.9
- Weight decay: 0.0005

- Optimizer: SGD
- Learning rate scheduler: StepLR (step size=25, gamma=0.1)
- Early stopping patience: 10 epochs

Loss Components

The total training loss was composed of four components: box regression loss, RPN box regression loss, classifier loss, and objectness loss. For logging purposes and comparison with YOLO results, these were grouped into box loss (loss_box_reg + loss_rpn_box_reg) and classification loss (loss_classifier + loss_objectness).

Evaluation Metrics

Validation was performed using COCO evaluation metrics, specifically tracking mAP@IoU=0.50 and mAP@IoU=0.50:0.95. The model's predictions were converted to COCO result format and evaluated using the pycocotools library. Early stopping was implemented based on mAP@IoU=0.50:0.95 to prevent overfitting.

Logging and Monitoring

Training progress was monitored using TensorBoard for real-time visualization of loss curves and metrics. Additionally, a CSV file in YOLO-compatible format was generated containing epoch-wise metrics including training/validation losses, mAP scores, recall, and learning rates. The model was saved periodically every 10 epochs, with the best performing model (based on validation mAP) saved separately.

The implementation was executed on hardware consisting of an AMD Ryzen 9 5950X CPU (16 cores/32 threads), 64 GB RAM, and an NVIDIA GeForce RTX 5070 Ti GPU with 16 GB VRAM, using CUDA 13.1 for acceleration.

4.1.3 RetinaNet

4.1.4 RF-DETR

4.2 YOLO-based Detectors

The implementation of different YOLO versions (YOLOv8, YOLOv11, and YOLOv12) followed a consistent methodology across all group members, with variations primarily in model size selection and hardware-specific optimizations. This section describes the common approach and highlights significant deviations.

Common Implementation Framework

All YOLO implementations utilized the Ultralytics library and followed a standardized workflow consisting of dataset preparation, model training, and evaluation phases.

Dataset Preparation

The dataset preparation phase was consistent across all implementations. Images were extracted from a compressed archive (`images.zip`) into the YOLO dataset directory structure. The `data.yaml` configuration file was programmatically updated to reflect absolute paths for the training, validation, and test splits on each user's system. This ensured portability across different development environments while maintaining consistent dataset access patterns.

Model Selection and Training Configuration

Different YOLO versions and model sizes were employed:

- YOLOv8: Nano variant (`yolov8n.pt`) for baseline experiments
- YOLOv11: Small variant (`yolo11s.pt`) for balanced performance
- YOLOv12: Multiple variants tested (Large, Medium, Small) to evaluate size-performance tradeoffs

The standard training configuration shared across implementations included:

- Epochs: 100 with early stopping patience of 20
- Image size: 640×640 pixels (default)
- Device: Automatic GPU selection when available, CPU fallback otherwise
- Checkpoint saving: Every 10 epochs
- Task: Object detection (`task='detect'`)
- Project output: Organized under `runs/` directory

Significant Deviations

Hardware-Constrained Optimization One implementation utilized reduced batch size (8) and image resolution (512×512) due to hardware memory limitations, demonstrating adaptive configuration for resource-constrained environments.

Advanced Optimizer Configuration An alternative implementation experimented with the Adam optimizer instead of the default SGD, along with disk caching (`cache='disk'`) and increased worker threads (`workers=8`) to optimize I/O performance.

Automated Batch Sizing The YOLOv12 experiments utilized automatic batch sizing (`batch=-1`) targeting 60% GPU utilization, allowing the framework to dynamically determine optimal batch sizes based on available VRAM.

Data Augmentation Experiments Extended experiments with YOLOv12 included systematic comparison of default training versus augmented training with modified hyperparameters:

- Horizontal/vertical flipping disabled (`fliplr=0.0, flipud=0.0`) to preserve sign orientation
- Mosaic augmentation maintained at full strength (`mosaic=1.0`)
- Mixup augmentation introduced (`mixup=0.1`)
- Rotation augmentation enabled (`degrees=10.0`)
- Scale variation applied (`scale=0.5`)

These augmentation strategies were designed specifically for traffic sign detection, where orientation preservation is critical but lighting and scale variations are beneficial.

Monitoring and Logging

All implementations enabled TensorBoard logging for real-time training visualization. Training metrics including loss curves, mAP scores, precision, and recall were logged automatically to CSV files in YOLO's standard format, facilitating cross-model comparison and performance analysis.

5 Evaluation of the object detectors

6 References and List of Resources Used

References

[1] A. Spiteri, D. Pace, E. R. Debrincat, and J. Grech, "Annotation Process Tools," GitHub repository, spiteriamy/ari3129-cv-group-project, Jan. 2026. [Online]. Available: `https://github.com/spiteriamy/ari3129-cv-group-project/tree/main/Annotation%20Process`

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[3] R. Girshick, "Fast R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1440–1448.

[4] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, USA, 2014, pp. 580–587.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.